Modul Pertemuan 4: Dasar Flutter

IF24502 Pemrograman Berbasis Platform · 2026.1 · Ilmu Informatika UKDC

Boleh dibaca, dicetak, dan dibagikan untuk belajar dengan atribusi; tidak boleh diubah atau dipakai komersial.

Baca sebelum kelas. Modul ini membuka Flutter. Bahan Dart yang dipakai di sini, terutama class dan inheritance (inheritance), ada di Modul Pertemuan 3.

1. Tujuan pembelajaran

Setelah membaca modul ini, Anda mampu:

1. Menjelaskan mengapa di Flutter **semuanya widget**, dan bahwa widget adalah **class Dart** biasa.
2. Membuat dan menjalankan proyek Flutter, serta membedakan **hot reload** dan **hot restart**.
3. Menggambar **widget tree** dari satu layar.
4. Memilih **StatelessWidget** atau **StatefulWidget**, dan memakai `setState` dengan benar.
5. Menyusun layar dengan `Column`, `Row`, `Expanded`, `Padding`, dan `ListView.builder`.

2. Semuanya widget, dan widget adalah class

Di Flutter, setiap bagian layar adalah **widget**: teks, tombol, jarak, baris, bahkan halaman dan aplikasinya sendiri. Tidak ada file layout terpisah seperti XML atau HTML. Layar disusun dengan menulis kode Dart.

Widget bukan hal baru. Ia adalah **class** yang memakai **inheritance** dari Pertemuan 3: class Anda `extends StatelessWidget` atau `extends StatefulWidget`, lalu **meng-override** satu method, `build`, yang mengembalikan widget lain.

```dart
class Sapaan extends StatelessWidget {
  const Sapaan({super.key});

  @override
  Widget build(BuildContext context) {
    return const Text('Halo, Guyub');
  }
}
```

Tiga hal yang sama dengan class di Pertemuan 3:

Di Pertemuan 3	Di widget
`class Story extends ItemBacklog`	`class Sapaan extends StatelessWidget`

Di Pertemuan 3	Di widget
`@override` pada method induk	`@override` pada `build`
Constructor dengan named parameter	`const Sapaan({super.key})`

`super.key` cukup ditulis apa adanya di setiap widget. Kegunaannya dibahas saat daftar mulai berubah urutan.

`const` muncul hampir di setiap baris kode Flutter: `const Text(...)`, `const Sapaan()`. Artinya object itu tidak pernah berubah, sehingga Flutter cukup membuatnya sekali dan tidak perlu membangunnya ulang. Aturan praktisnya: tulis `const` bila `flutter analyze` menyarankannya, dan jangan pakai `const` pada widget yang isinya berasal dari variabel yang berubah.

3. Proyek pertama dan hot reload

3a. Membuat dan menjalankan

Latihan Flutter tinggal di repository studio Anda, dalam folder pertemuannya. Di terminal:

```
cd pbp-studio-<nim>
mkdir p04-studio
cd p04-studio
flutter create guyub
cd guyub
flutter run -d chrome
```

Jangan menjalankan `git init` di dalam folder `guyub`. Folder itu sudah berada di dalam repository studio, jadi cukup di-commit dari sana.

`-d chrome` menjalankan di browser. Ganti dengan `-d macos`, `-d windows`, atau nama emulator Android dari `flutter devices`. Kode yang sama jalan di semuanya; itulah inti mata kuliah ini.

File yang penting untuk sekarang hanya dua:

File	Isinya
`lib/main.dart`	Kode aplikasi. Program mulai dari fungsi `main()`
`pubspec.yaml`	Nama proyek, versi, dan paket tambahan yang dipakai

3b. Hot reload dan hot restart

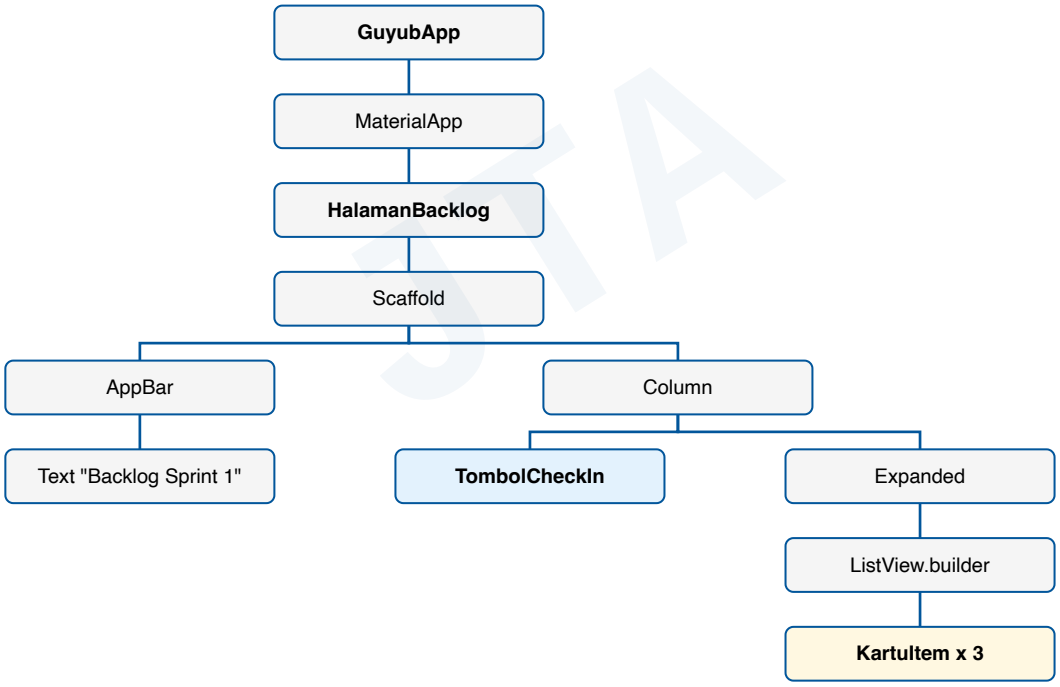
Selama `flutter run` berjalan, simpan perubahan lalu tekan di terminal:

Tombol	Nama	Yang terjadi	State (misalnya status check-in)
`r`	**Hot reload**	Kode baru disuntikkan, `build` dijalankan ulang, kurang dari satu detik	**Tetap**
`R`	**Hot restart**	Aplikasi dimulai ulang dari `main()`	**Kembali ke awal**

Pakai hot reload untuk mengubah tampilan. Pakai hot restart bila mengubah `main()`, nilai awal sebuah variabel state, atau bila hasil hot reload terasa aneh.

4. Widget tree

Widget disusun bersarang: satu widget membungkus widget lain lewat `child` (satu anak) atau `children` (banyak anak). Susunan itu disebut **widget tree**.



Gambar 1. Widget tree layar backlog Guyub. Baca dari atas ke bawah: setiap kotak adalah satu widget, dan garis menunjuk ke widget yang dibungkusnya. Empat kotak bercetak tebal adalah class yang kita tulis sendiri; sisanya bawaan Flutter. TombolCheckIn berwarna biru karena hanya dia yang stateful. KartuItem berwarna kuning karena muncul berulang, satu per item backlog.

Layar pada Gambar 1 ditulis seperti ini. Class `ItemBacklog` di sini versi ringkas dari Pertemuan 3: semua field `final`, dan constructor-nya `const` dengan named parameter.

```dart
import 'package:flutter/material.dart';

void main() {
  runApp(const GuyubApp());
}

class ItemBacklog {
  final String kunci;
  final String judul;
  final int poin;

  const ItemBacklog({required this.kunci, required this.judul, required this.poin});
}

const daftarItem = [
  ItemBacklog(kunci: 'GYB-001', judul: 'Halaman masuk', poin: 3),
  ItemBacklog(kunci: 'GYB-002', judul: 'Tombol check-in', poin: 5),
  ItemBacklog(kunci: 'GYB-003', judul: 'Rekap kehadiran mingguan', poin: 8),
];

class GuyubApp extends StatelessWidget {
  const GuyubApp({super.key});

  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      title: 'Guyub',
      home: HalamanBacklog(),
    );
  }
}

class HalamanBacklog extends StatelessWidget {
  const HalamanBacklog({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Backlog Sprint 1')),
      body: Column(
        children: [
          const TombolCheckIn(),
```

```
          Expanded(
            child: ListView.builder(
              itemCount: daftarItem.length,
              itemBuilder: (context, i) => KartuItem(item: daftarItem[i]),
            ),
          ),
        ],
      ),
    );
  }
}
```

Cara membaca widget tree dari kode: setiap `child:` dan `children:` adalah satu branch ke bawah. `MaterialApp` memberi gaya Material, `Scaffold` memberi kerangka halaman (bilah atas dan isi), dan `body` berisi `Column` yang menyusun anak-anaknya dari atas ke bawah.

5. Stateless dan stateful

5a. StatelessWidget: tampilannya hanya bergantung pada masukan

`KartuItem` menampilkan satu item. Ia tidak pernah berubah sendiri; kalau datanya berganti, induknya membuat `KartuItem` baru.

JTA

```dart
class KartuItem extends StatelessWidget {
  final ItemBacklog item;

  const KartuItem({super.key, required this.item});

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(8),
      child: Row(
        children: [
          Text(item.kunci),
          const SizedBox(width: 12),
          Expanded(child: Text(item.judul)),
          Text('${item.poin} poin'),
        ],
      ),
    );
  }
}
```

5b. StatefulWidget: ada yang berubah karena pengguna

Tombol check-in harus ingat apakah pengguna sudah hadir. Yang diingat itu disebut **state**. StatefulWidget ditulis sebagai **dua class**: widgetnya, dan class `State` yang menyimpan state serta berisi `build`.

```dart
class TombolCheckIn extends StatefulWidget {
  const TombolCheckIn({super.key});

  @override
  State<TombolCheckIn> createState() => _TombolCheckInState();
}

class _TombolCheckInState extends State<TombolCheckIn> {
  bool sudahHadir = false;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(8),
      child: Row(
        children: [
          Text(sudahHadir ? 'Sudah check-in' : 'Belum check-in'),
          const Spacer(),
          ElevatedButton(
            onPressed: () {
              setState(() {
                sudahHadir = !sudahHadir;
              });
            },
            child: Text(sudahHadir ? 'Check-out' : 'Check-in'),
          ),
        ],
      ),
    );
  }
}
```

`setState` memberi tahu Flutter bahwa state berubah, sehingga `build` dijalankan ulang dan layar diperbarui. Mengubah `sudahHadir` tanpa `setState` mengubah nilainya, tetapi layar tidak tahu dan tidak berganti.

Garis bawah di depan `_TombolCheckInState` membuat class itu **private** untuk file ini. Hanya `TombolCheckIn` yang boleh memakainya.

5c. Aturan memilih

Mulai dari stateless. Naikkan ke stateful hanya kalau ada nilai yang berubah karena pengguna atau waktu, dan nilai itu milik widget ini sendiri.

Widget	Pilihan	Alasan
Judul halaman	Stateless	Tidak pernah berubah
Satu kartu item	Stateless	Berubah hanya kalau induknya memberi data baru
Tombol check-in	Stateful	Status hadir berubah karena ditekan
Seluruh halaman	Stateless	Bagian yang berubah sudah dipisah ke `TombolCheckIn`

Membuat semuanya stateful "supaya aman" justru membuat lebih banyak bagian layar dibangun ulang dan lebih sulit dibaca. Pada Gambar 1, hanya satu kotak yang biru.

6. Widget layout dasar

Widget	Gunanya	Ingat
`Column`	Menyusun anak dari atas ke bawah	`children:`
`Row`	Menyusun anak dari kiri ke kanan	`children:`
`Expanded`	Membuat satu anak `Row` atau `Column` mengambil sisa ruang	Hanya boleh di dalam `Row`, `Column`, atau `Flex`
`Padding`	Memberi jarak di sekeliling satu anak	`EdgeInsets.all(8)`
`SizedBox`	Jarak tetap, atau kotak berukuran tetap	`SizedBox(width: 12)`
`Spacer`	Ruang kosong yang mendorong anak berikutnya ke ujung	Di dalam `Row` atau `Column`
`ListView.builder`	Daftar yang bisa digulir, membuat item hanya saat terlihat	`itemCount` dan `itemBuilder`

Pada `KartuItem`, `Expanded` membungkus judul supaya judul yang panjang mengambil ruang tengah dan poin tetap di kanan. Pada `HalamanBacklog`, `Expanded` membungkus `ListView` supaya daftar mendapat sisa tinggi layar di bawah tombol check-in.

7. Empat error Flutter yang paling sering

Pesan error di bawah dikutip apa adanya dari Flutter 3.47.

1. `setState` di StatelessWidget.

```
class Hitung extends StatelessWidget {
  const Hitung({super.key});

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: () {
        setState(() {});
      },
      child: const Text('Tambah'),
    );
  }
}
```

```
error • The method 'setState' isn't defined for the type 'Hitung'. Try correcting the
name to the
name of an existing method, or defining a method named 'setState' •
lib/hitung.dart:10:9 •
undefined_method
```

StatelessWidget tidak punya state, jadi tidak punya `setState`. Ubah menjadi StatefulWidget.

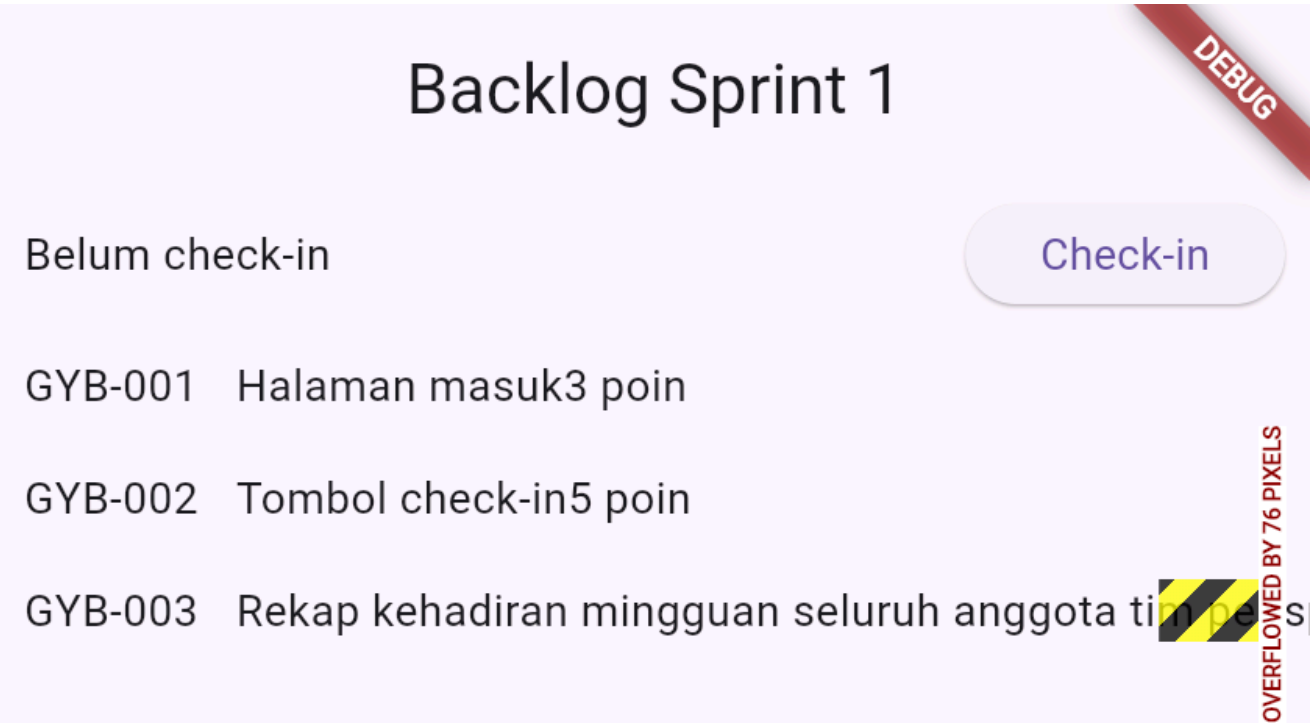
2. `ListView` di dalam `Column` tanpa `Expanded`.

```
Column(
  children: [
    const Text('Backlog'),
    ListView(children: const [Text('GYB-001')]),
  ],
)
```

```
Vertical viewport was given unbounded height.
```

`Column` memberi anaknya tinggi tak terbatas, sedangkan `ListView` ingin mengisi seluruh tinggi yang ada. Bungkus `ListView` dengan `Expanded`.

3. Pita kuning-hitam bertuliskan "OVERFLOWED BY ... PIXELS". Isi `Row` lebih lebar dari layar. Bungkus teks yang panjang dengan `Expanded`.



Gambar 2. Kartultem yang judulnya ditulis Text(item.judul) tanpa Expanded. Baris ketiga terlalu panjang, sehingga Flutter menggambar pita kuning-hitam di kanan. Baris pertama dan kedua juga salah: tanpa Expanded, poin menempel ke judul dan tidak terdorong ke kanan. Pita hanya tampil saat aplikasi dijalankan dalam mode debug.

4. Hot reload tidak mengubah apa-apa. Yang diubah adalah `main()` atau nilai awal state. Pakai hot restart (`R`).

8. Hubungan dengan materi lain

- **Asynchronous programming** (Modul Pertemuan 3 bagian 7) dipakai lagi saat aplikasi mengambil data dari server: data datang belakangan, dan layar harus menampilkan tanda memuat.
- **Git dan GitHub** (Modul Pertemuan 3 bagian 8): latihan Flutter di-commit di repository studio; proyek kelompok kelak punya repository sendiri. `flutter create` sudah membuat `.gitignore` yang membuang folder `build/`.

Glosarium

Istilah	Arti
Widget	Class Dart yang menggambarkan satu bagian layar
Widget tree	Susunan widget yang saling membungkus, dari aplikasi sampai teks terkecil
`build`	Method yang mengembalikan widget anak; dijalankan ulang setiap kali tampilan perlu diperbarui

Istilah	Arti
StatelessWidget	Widget yang tampilannya hanya bergantung pada masukan dari induknya
StatefulWidget	Widget yang menyimpan state sendiri di class `State`
State	Nilai yang bisa berubah selama aplikasi berjalan dan memengaruhi tampilan
`setState`	Memberi tahu Flutter bahwa state berubah, supaya `build` dijalankan ulang
Layout	Cara widget disusun di layar: `Row`, `Column`, `Expanded`, `ListView`
Hot reload	Menyuntikkan kode baru tanpa menghapus state
Hot restart	Memulai ulang aplikasi dari `main()`; state kembali ke awal
Private	Hanya bisa dipakai di file yang sama; ditandai garis bawah di depan nama
Inheritance	Inheritance: class baru `extends` class lain dan mendapat semua isinya
Override	Menimpa method milik class induk; ditandai `@override`
Constructor	Cara membuat object dari sebuah class
`const`	Object yang tidak pernah berubah, sehingga cukup dibuat sekali
`child`, `children`	Satu widget anak, dan daftar widget anak
`MaterialApp`	Widget akar yang memberi gaya Material ke seluruh aplikasi
`Scaffold`	Kerangka satu halaman: bilah atas dan isi
Overflow	Isi lebih besar dari ruang yang tersedia; Flutter menandainya dengan pita kuning-hitam
Emulator	Perangkat Android tiruan yang berjalan di laptop

Catatan